Documento Funcional — AgendaPet Web

Painel de agendamentos do pet shop Patinhas & Cia — o que construir, para quem, e como deve parecer

Desenvolvimento Front-End [React] T4 | Módulo 1 | Semanas 13 e 14 | Projeto: **Mini-Projeto Avaliativo M1.2 (AgendaPet Web)**
Leia este documento **primeiro**: ele diz *o quê* e *por quê*. O documento **Refinamento Técnico** diz *como* (arquivos, funções, contratos de dados e modelos de código). O **enunciado oficial** (.docx) traz o prazo, a pontuação e o checklist de entrega.

---

1. Visão geral

Item	Descrição
Produto	**AgendaPet Web** — painel web (HTML + CSS + JavaScript puros, sem bibliotecas) + scripts de terminal em Node.js
Cliente	**Patinhas & Cia**, pet shop de bairro (banho, tosa, consulta, vacina, adestramento e hospedagem), empresa fictícia
Problema-âncora	Os agendamentos são feitos por WhatsApp e caderno: horários marcados duas vezes, atendimentos em feriados sem aviso, endereços anotados errado nas visitas a domicílio e nenhuma noção de quanto o dia rendeu.
Por que isso importa	Um(a) dev júnior recebe um pedido real de cliente, um backlog e uma sprint. Este projeto simula essa rotina: **planejar, estimar, entregar por tickets, revisar, depurar e apresentar**.
Como o trabalho acontece	**Sprint de 2 semanas** (16/10 a 02/11/2026): Sprint Planning → um ticket por branch com Pull Request → Sprint Review (apresentação em live coding no Google Meet) → Retrospectiva (README).
Resultado esperado	Atendente agenda com validação e vê a agenda do dia; dona da loja vê o fechamento; o mesmo cálculo roda no **terminal** com Node.js.

2. Quem usa

Persona	Contexto	O que precisa
Bia, atendente	Trabalha em pé na bancada, com o **celular ou tablet** numa mão e muitas vezes com a outra ocupada segurando um pet	Botões grandes; formulário curto e sem erros bobos; ver rápido a agenda do dia; ser avisada de atendimentos próximos
Seu Henrique, dono da loja	Fecha o caixa no fim do dia e precisa confiar nos números	Faturamento só do que foi **concluído**; quanto ainda vai receber; quantos atendimentos por serviço; relatório também no terminal

Rafael, atendente que usa só o teclado	Navega com Tab, Enter e Espaço, às vezes com leitor de tela	Tudo alcançável por teclado, foco bem visível, erros anunciados, atalhos (`N`, `/`, `Esc`)

3. Escopo

Nível	O que entra
Caminho-base (obrigatório)	Página semântica e acessível · catálogo de serviços por `fetch` com 3 estados · formulário com validação · endereço por CEP · alerta de feriados com cache · agenda do dia com busca, filtro e atalhos · regras de negócio com classes e herança · persistência no `localStorage` · relógio e lembretes · layout responsivo · projeto Node.js com ESM e CommonJS · depuração com relatório de bug
⭐ **Bônus** (até +0,5)	Publicar no GitHub Pages · testes com `node --test` · baixar a agenda como arquivo (`Blob`) · estilos de impressão (`@media print`)
Fora do escopo	Login, banco de dados em servidor, pagamento, envio real de WhatsApp/SMS, vários profissionais ou várias lojas, bibliotecas e frameworks

4. Requisitos funcionais (Product Backlog)

Cada requisito é uma **história de usuário** (US). Os cenários completos de aceite estão no item 5.

ID	História (Como… quero… para…)	Regras principais	Prior.
US01	Como atendente, quero uma página organizada e navegável por teclado, para operar a agenda com rapidez e sem erros.	Landmarks (`header` , `nav` , `main` , `section` , `aside` , `footer`); um só `h1` ; `lang` , `title` , `meta description` ; skip link; `alt` nas imagens	Alta
US02	Como atendente, quero ver os serviços com preço e duração, para informar o tutor e escolher no agendamento.	`data/servicos.json` (≥ 6 serviços) lido com `fetch` + `async/await` ; estados carregando, erro (com "Tentar novamente") e sucesso	Alta
US03	Como atendente, quero registrar um agendamento com validação, para não perder dados nem marcar horários inválidos.	Campos obrigatórios; telefone com DDD; data não passada; horário dentro do funcionamento; erros na tela (sem `alert`) e foco no 1º inválido	Alta
US04	Como atendente, quero informar só o CEP em atendimento a domicílio, para que o endereço seja preenchido sem erros.	ViaCEP ao completar 8 dígitos; CEP inválido, inexistente e falha de rede com mensagens diferentes; preenchimento manual sempre possível	Média
US05	Como atendente, quero ser avisada quando a data for feriado, para confirmar com o tutor antes de agendar.	BrasilAPI por ano; cache no `localStorage` com validade; "Agendar mesmo assim" obrigatório; API fora do ar não bloqueia	Média
US06	Como atendente, quero ver, buscar e atualizar os agendamentos do dia, para acompanhar o fluxo da loja.	Lista por horário; busca em tempo real; filtro por status; concluir/cancelar; detalhes expansíveis; atalhos `N`, `/`, `Esc`; realce no mouse	Alta
US07	Como dona do pet shop, quero que o sistema calcule valores, impeça conflitos e feche o dia, para saber quanto a loja faturou.	Classes com herança (taxa domiciliar); conflito de horário; protocolo sequencial (closure); callbacks; fechamento com métodos de array	Alta
US08	Como atendente, quero que a agenda continue ali ao recarregar a página, para não perder o trabalho do dia.	`localStorage` (agenda, filtro, tema) com `try/catch` ; "Limpar meus dados" só das chaves do projeto; exportar JSON	Alta
US09	Como atendente, quero ser lembrada dos próximos atendimentos, para receber os tutores sem atrasar.	`setInterval` (30 s) para relógio e lembretes; `setTimeout` para o toast; lembrete com ≤ 15 min; "Atrasado"; pausar/retomar	Média
US10	Como atendente, quero usar o painel no celular ou tablet na bancada, com letras legíveis e botões fáceis de tocar.	Flexbox; mobile-first; 360/768/1280 px; variáveis CSS; sem `!important` e sem seletores de id; `:focus-visible` ; contraste ≥ 4,5:1	Alta
US11	Como dona do pet shop, quero fechar o dia pelo terminal e validar os dados exportados, para conferir o caixa fora do navegador.	`package.json` com scripts; `devDependency` local; ESM (`fechamento-dia.js`) e CommonJS (`validar-agenda.cjs`); README: global vs local e CJS vs ESM	Alta

US12	Como pessoa desenvolvedora, quero investigar um defeito com o depurador, para corrigir a causa e não só o sintoma.	`.vscode/launch.json`; breakpoint, condicional/logpoint, Watch e Call Stack; `docs/bug-report.md` com evidências	Média

5. Cenários de aceite (Dado · Quando · Então)

Um ticket só está **pronto** quando todos os cenários da história passam rodando o projeto a partir de um clone limpo.

US	Cenário
US01	**Dado** a página aberta, **quando** navego só com `Tab`, **então** o primeiro foco é o "Ir para o conteúdo" e todo controle recebe foco visível. **Dado** o HTML, **então** há um único `h1`, títulos sem saltos, `lang="pt-BR"`, `title` e `meta description`.
US02	**Dado** o servidor local, **quando** a página abre, **então** aparece "carregando" e depois os 7 serviços com preço e duração. **Dado** o caminho do JSON quebrado (aponte o `fetch` para um arquivo que não existe), **então** aparece erro amigável com "Tentar novamente" e nenhuma exceção no console.
US03	**Dado** o formulário vazio, **quando** envio, **então** aparecem mensagens na tela (sem `alert`) e o foco vai ao primeiro campo inválido. **Dado** data passada ou horário fora do funcionamento, **quando** envio, **então** não salva e a mensagem explica o motivo. **Dado** dados válidos, **quando** envio, **então** a página não recarrega, mostra confirmação e o item entra na agenda.
US04	**Dado** o CEP 01001-000, **quando** completo os 8 dígitos, **então** rua, bairro, cidade e UF são preenchidos. **Dado** CEP inexistente, formato curto ou sem rede, **então** cada caso mostra uma mensagem diferente e o preenchimento manual continua possível.
US05	**Dado** o dia 25/12, **quando** escolho a data, **então** vejo o nome do feriado e só salvo após marcar "Agendar mesmo assim". **Dado** que consultei o ano uma vez, **quando** escolho outra data, **então** não há nova requisição (cache). **Dado** a API indisponível, **então** aparece aviso e posso continuar.
US06	**Dado** vários agendamentos, **então** a lista está ordenada por horário, com contador e mensagem para lista vazia. **Dado** a busca ativa, **quando** digito "sonia", **então** filtra em tempo real (ignora acento); `/` foca a busca, `Esc` limpa, `N` leva ao formulário (e nada disso dispara enquanto digito em um campo). **Dado** um item, **quando** clico em Concluir ou Cancelar, **então** o status muda sem recarregar.
US07	**Dado** um agendamento domiciliar, **então** o valor inclui a taxa de deslocamento e o resumo mostra o endereço. **Dado** um horário ocupado, **quando** tento salvar outro ativo nele, **então** o sistema impede e informa o conflito; se o primeiro for cancelado, o horário é liberado. **Dado** três agendamentos, **então** os protocolos são AP-0001, AP-0002, AP-0003. **Dado** o fechamento, **então** o faturamento soma só os concluídos e há contagem por serviço e de cancelados.
US08	**Dado** agendamentos salvos, **quando** recarrego (F5), **então** agenda, filtro e tema são restaurados. **Dado** um valor corrompido no `localStorage`, **então** a página abre normalmente. **Dado** "Limpar meus dados", **então** só as chaves `agendapet:*` são removidas.
US09	**Dado** um atendimento hoje, **então** o texto "próximo atendimento em X min" atualiza sozinho. **Dado** um atendimento atrasado ou a 15 min ou menos, **então** há destaque ou lembrete visual. **Dado** "Pausar lembretes", **então** a atualização para (`clearInterval`) e volta ao retomar, sem timers duplicados.

US10	**Dado** 360, 768 e 1280 px de largura, **então** não há rolagem horizontal e o layout se reorganiza. **Dado** o CSS, **então** há Flexbox, variáveis, 2 ou mais breakpoints, `:focus-visible`, nenhum `!important` e nenhum seletor de id.
US11	**Dado** um clone limpo, **quando** rodo `npm install` e `npm start`, **então** a página abre pelo servidor local. **Dado** `npm run fechamento`, **então** o terminal imprime o fechamento usando os módulos de `js/dominio/`. **Dado** `npm run validar:erro`, **então** o script lista os erros e termina com código 1.
US12	**Dado** o VS Code, **quando** pressiono F5, **então** o script de Node inicia pelo `launch.json` e para no breakpoint. **Dado** `docs/bug-report.md`, **então** constam passos, causa raiz, evidência e correção ligada a um commit/PR.

6. Regras de negócio

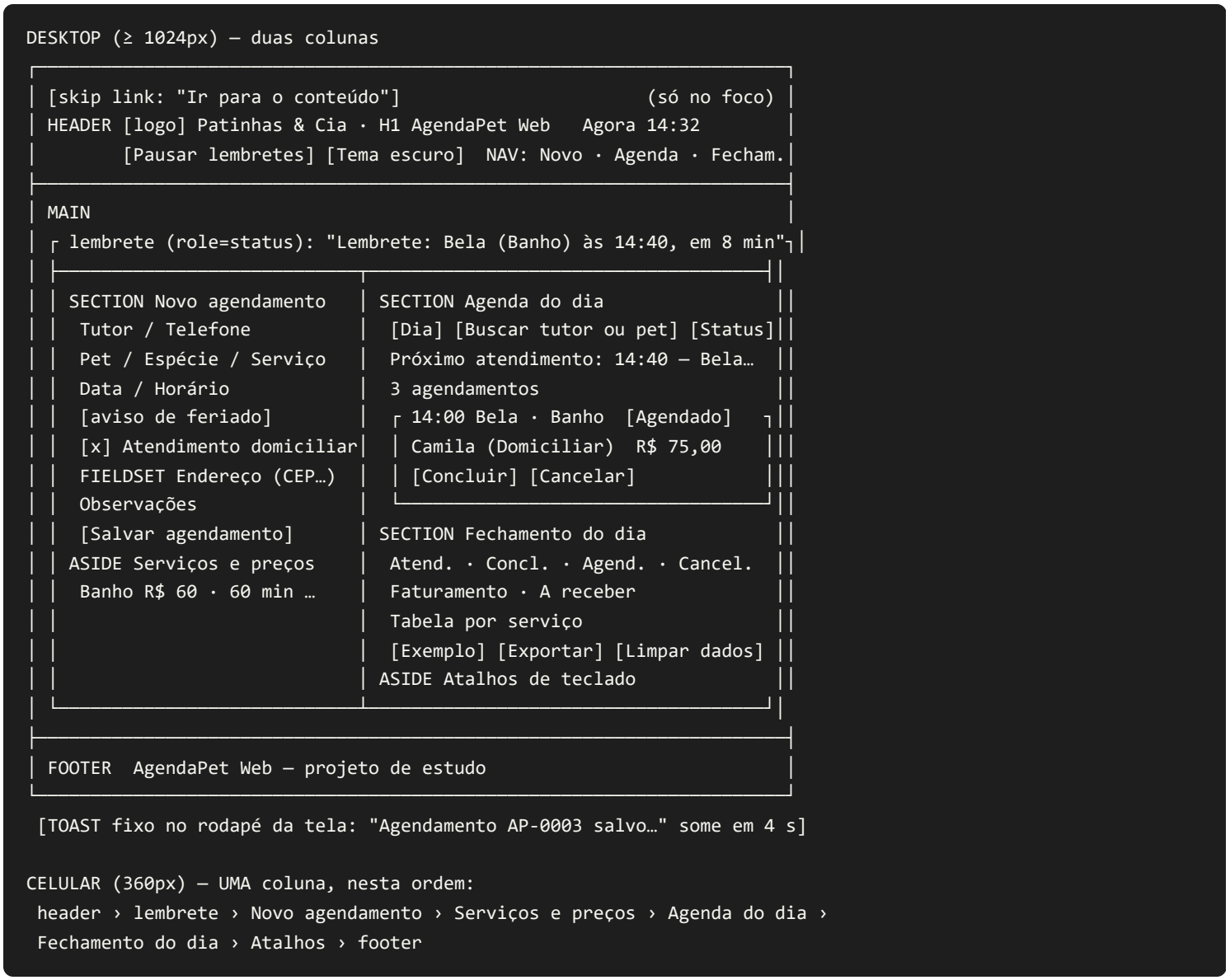
ID	Regra
RN01	**Funcionamento:** das 08:00 às 18:00, horários de 30 em 30 minutos. O atendimento precisa **terminar** até 18:00 (banho e tosa de 120 min não pode começar depois de 16:00).
RN02	**Conflito:** não pode haver dois agendamentos **ativos** (Agendado ou Concluído) no mesmo dia e horário. **Cancelado libera** o horário.
RN03	**Domicílio:** só serviços marcados como domiciliares (adestramento e hospedagem são só na loja). Taxa de deslocamento: **R$ 15** se a casa é em São Paulo/SP; **R$ 30** em outra cidade. *(Você pode adotar outro critério de taxa, desde que registre no README.)*
RN04	**Valor do agendamento:** preço do serviço (+ taxa, se domiciliar).
RN05	**Status:** nasce **Agendado**; pode virar **Concluído** ou **Cancelado**. Concluído e Cancelado ficam travados (botões desabilitados).
RN06	**Fechamento do dia:** faturamento = soma dos **Concluídos**; "a receber" = soma dos **Agendados**; cancelados não entram em valor; "por serviço" conta os ativos. O dia está **encerrado** quando não resta nenhum Agendado.
RN07	**Protocolo:** `AP-0001`, `AP-0002` … sequencial, sem repetir; continua do maior número já salvo.
RN08	**Feriado:** em feriado nacional é preciso confirmar "Agendar mesmo assim". Se não foi possível verificar, o sistema avisa e **não** bloqueia.
RN09	**Lembrete:** quando faltarem 15 minutos ou menos para o próximo atendimento de hoje. **Atrasado** = ainda Agendado e o horário já passou.

7. Requisitos não funcionais

Tema	Requisito
Acessibilidade	Estrutura semântica, `label for`, `aria-live` / `role="alert"` para mensagens, foco visível, skip link, contraste mínimo 4,5:1, informação nunca só pela cor. Meta: Lighthouse de Acessibilidade e SEO ≥ 90

Responsividade	Mobile-first; sem rolagem horizontal em 360, 768 e 1280 px; alvos de toque ≥ 44 px
Resiliência	Nenhuma falha de API ou de `localStorage` pode "quebrar" a página: sempre há mensagem e caminho alternativo
Segurança básica	Texto digitado entra na tela com `textContent` (nunca `innerHTML`); nenhuma chave, token ou dado pessoal real no repositório
Dados no navegador	Só chaves com prefixo `agendapet:`; "Limpar" não pode apagar dados de outros sistemas
Qualidade de código	Regra de negócio em `js/dominio/` (sem DOM); `const` / `let` (nunca `var`); funções pequenas; nomes claros; sem código morto
Compatibilidade	Navegadores atuais (Chrome, Edge, Firefox); Node.js 20 ou superior
Privacidade	Os dados de agendamento ficam apenas no navegador da pessoa usuária; as consultas externas enviam só o CEP e o ano

8. Mapa da tela (wireframe)



9. Estados da interface e textos (microcopy)

Situação	Tipo	Texto exibido	Some sozinho?
Carregando o catálogo	carregando	"Carregando serviços…"	Some ao concluir
Falha ao carregar o catálogo	erro	"Não foi possível carregar os serviços (erro HTTP 404)." + botão "Tentar novamente"	**Não**

Sem rede no catálogo	erro	"Sem conexão para carregar os serviços." + botão "Tentar novamente"	**Não**
Erro de validação (por campo)	erro	"Informe o nome do tutor (mínimo 3 letras)." · "Informe um telefone com DDD (10 ou 11 dígitos)." · "Não é possível agendar em uma data que já passou." · "Fora do funcionamento: atendemos das 08:00 às 18:00, em horários de 30 em 30 minutos, e o serviço precisa terminar até o fechamento." · "Esse horário de hoje já passou."	**Não** (até corrigir)
Conflito de horário	erro	"Já existe um atendimento às {hora} em {dd/mm/aaaa}. Escolha outro horário."	**Não**
Domiciliar indisponível	informativo	""{serviço}" é realizado somente na loja."	Troca ao mudar o serviço
Buscando CEP	carregando	"Buscando endereço…"	É substituído
CEP encontrado	sucesso	"Endereço preenchido. Confira e informe o número."	Não
CEP com formato inválido	erro	"O CEP deve ter 8 dígitos."	**Não**
CEP não encontrado	erro	"CEP não encontrado. Preencha o endereço manualmente."	**Não**
Sem rede / serviço de CEP fora	erro	"Sem conexão com o serviço de CEP. Preencha o endereço manualmente." · "O serviço de CEP está indisponível agora. Preencha o endereço manualmente."	**Não**
Data é feriado	alerta	"25/12/2026 é feriado: Natal. Confirme com o tutor antes de agendar." + caixa "Agendar mesmo assim"	Some ao trocar a data
Feriado sem confirmar	erro	"Marque "Agendar mesmo assim" para agendar em um feriado."	**Não**
Feriados indisponíveis	alerta	"Não foi possível verificar se a data é feriado agora. Você pode continuar o agendamento."	Some ao trocar a data
Agendamento salvo	sucesso (toast)	"Agendamento AP-0001 salvo: Thor, 21/10/2026 às 09:00."	**Sim, em 4 s**
Concluído / cancelado	sucesso / info (toast)	"AP-0001 (Thor) concluído." · "AP-0001 (Thor) cancelado. O horário foi liberado."	**Sim, em 4 s**
Lembretes pausados / retomados	info (toast)	"Lembretes pausados." · "Lembretes retomados."	**Sim, em 4 s**
Lembrete de atendimento	alerta	"Lembrete: Bela (Banho) às 14:40, em 8 min."	Some quando passa de 15 min ou ao pausar

Lista vazia	informativo	"Nenhum agendamento para este dia. Use o formulário para criar o primeiro." · "Nenhum agendamento corresponde à busca ou ao filtro."	Some ao haver itens
Limpar dados (2 etapas)	botão	"Limpar meus dados" → "Clique de novo para confirmar" (volta em 4 s) → toast "Dados da AgendaPet removidos (N itens)."	Botão volta sozinho

> **Regra de acessibilidade**
> Mensagem de erro **não desaparece sozinha** e **não depende só da cor**: sempre tem texto, e o campo fica com borda destacada e `aria-invalid="true"`. Toasts usam `role="status"` (anunciados sem roubar o foco).

10. Identidade visual

10.1 Paleta (tokens CSS)

Todas as cores moram em **variáveis CSS** no `:root`; o tema escuro só redefine as variáveis. Você pode criar a sua identidade, desde que mantenha o **contraste mínimo de 4,5:1** para texto.

--primaria #00695c botões, títulos	**--primaria-escura** #004d40 hover	**--acento** #ffb300 atrasado, lembrete (nunca texto)	**--fundo** #f3f7f6 fundo da página	**--superficie-2** #e8f1ef blocos internos
--texto #1c2b2d texto principal	**--erro** #b3261e erros, cancelar	**--sucesso** #1b5e20 concluído	**--info** #0d47a1 agendado, próximo	**--foco** #0b57d0 anel de foco

10.2 Tipografia e espaçamento

- **Fonte:** pilha do sistema (`"Segoe UI", Roboto, "Helvetica Neue", Arial, sans-serif`) — nada para baixar.
- **Tamanhos:** texto 16 px; títulos de cartão 19 px; horário do item 18 px; textos secundários 14 px (nunca menos que 12,8 px).
- **Espaçamento:** unidade de 16 px (`--espaco`); cantos arredondados de 10 px (`--raio`).

10.3 Logo e ícones

Marca simples em **SVG** (patinha branca em círculo verde) em `img/logo.svg` e `img/favicon.svg`. O logo é decorativo ao lado do nome da loja: `alt=""`.

10.4 Componentes

Componente	Como deve ser
Botão principal / secundário / perigo	Altura mínima 44 px; principal = fundo verde; secundário = borda verde; perigo = borda vermelha. Desabilitado: opacidade reduzida e cursor "não permitido"
Campo de formulário	Rótulo sempre visível acima; altura mínima 44 px; borda vermelha de 2 px + mensagem abaixo no erro

Cartão de agendamento	Borda lateral colorida pelo status (verde = agendado/concluído, vermelho = cancelado, âmbar = atrasado); hora em destaque; selo de status; valor à direita; detalhes expansíveis
Selo de status	Pílula com cor **e** texto ("Agendado", "Concluído", "Cancelado", "Atrasado")
Toast	Fixo no rodapé da tela; cor por tipo; **não bloqueia cliques**; some em 4 s
Tabela do fechamento	Cabeçalho com `th scope="col"` e legenda para leitor de tela

10.5 Princípios visuais

- Quem atende está com pressa: **poucos cliques**, textos curtos e botões grandes.
- Estado nunca só por cor: sempre há texto ou ícone junto.
- O tema escuro deve ser tão legível quanto o claro (confira o contraste nos dois).

11. Plano da sprint e entregáveis

Dia	Cerimônia	Tickets	Pronto quando
Sex 16/10	Liberação e Sprint Planning	—	Repositório, Kanban com 12 cartões estimados, `planejamento/sprint-planning.md`, branch `chore/setup-projeto`
Ter 20/10	Semana 13	US01, US10 (base), US02	Página semântica, layout responsivo e catálogo por `fetch` com 3 estados
Qui 22/10	Semana 13	US03, US04, US05	Formulário validado; CEP e feriados funcionando, com falhas tratadas
Sex 23/10	Semana 13	US06, US07	Agenda do dia, classes, herança, conflito e fechamento (testados no Node)
Ter 27/10	Semana 14	US08, US09	Dados sobrevivem ao F5; relógio e lembretes com pausa
Qui 29/10	Semana 14	US11, US12	Scripts npm, ESM e CommonJS; depuração e relatório de bug
Sex 30/10	Review e Retrospectiva	Ajustes	Acessibilidade, 360 px, README com retrospectiva; apresentação em live coding no Google Meet (15 min por estudante, turma reunida)
02/11, 22h	Entrega no AVA	—	Projeto entregue no AVA (links do repositório GitHub e do Kanban) e código publicado no GitHub

Entregáveis: repositório público com `index.html`, `css/`, `js/` (incluindo `js/dominio/`), `data/`, `scripts/`, `package.json` + `package-lock.json`, `.vscode/launch.json`, `.github/pull_request_template.md`, `planejamento/sprint-planning.md`, `docs/` (Kanban, depuração, relatório de bug) e `README.md`; link do Kanban; apresentação em live coding no Google Meet (Sprint Review, 15 minutos por estudante). A entrega precisa estar no AVA **e** no GitHub.

Definição de Pronto (para cada ticket)

- ✓ Os cenários de aceite da história passam
- ✓ Código em uma branch, commits no padrão `tipo(US##): descrição no imperativo`, Pull Request integrado à main
- ✓ Sem erros no console; layout conferido em 360 px; teclado conferido
- ✓ Cartão do Kanban em "Concluído" com o link do Pull Request
- ✓ Você consegue explicar linha a linha o que entregou

Rubrica de avaliação (resumo; a oficial está no enunciado)

Critério	Pontos	Evidência
01 Versionamento	0,5	≥ 5 branches `tipo/US##` , ≥ 12 commits no padrão, ≥ 5 Pull Requests com o modelo preenchido
02 Arquivos e README	0,5	Estrutura do projeto, `.gitignore`, README com decisões, global vs local, CJS vs ESM e retrospectiva
03 Apresentação em live coding (Sprint Review)	1,0	Todos os itens do 4.6 do enunciado, até 15 min, código e tela funcionando
04 Kanban e planejamento	0,5	4 colunas, 12 cartões estimados, links de branch e PR, `sprint-planning.md`
05 HTML, formulário, a11y, SEO	1,0	US01 e US03
06 CSS	1,0	US10: Flexbox, especificidade, responsividade
07 DOM e eventos	1,0	US06: seletores, mouse, teclado, busca, atalhos
08 fetch	1,0	US02, US04, US05: 3 estados e erros distintos
09 localStorage e timers	1,0	US05, US08, US09
10 Regras de negócio e POO	1,0	US07: classes, herança, closure, callback, arrays
11 Node.js	1,0	US11: npm, ESM, CommonJS, global vs local
12 Depuração	0,5	US12: `launch.json`, `bug-report.md` e demonstração